

PRAKTIKUM SISTEM OPERASI

MODUL 6



Oleh:

Resita Istantia Purwanto

2311104037

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL/TP

1. [10 Poin] Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah `$cat /proc/sys/kernel/pid_max`

Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!

Jawaban :Dalam sistem operasi Xinu, batasan maksimal proses tidak ditentukan secara dinamis seperti Linux, melainkan didefinisikan secara statis melalui sebuah makro (*macro constant*) di dalam file *header* konfigurasi.

- Lokasi Source Code & Nilai Maksimal Default

Batasan ini dapat ditemukan pada file source code: `include/kernel.h` (atau pada beberapa versi Xinu terletak di `sys/h/conf.h` atau `include/process.h`). Secara default, makro yang mengatur jumlah maksimal proses adalah `NPROC`.

- Maksimal proses default dalam Xinu: Biasanya diatur sebanyak 8 atau 16 proses (tergantung pada arsitektur target seperti Backend MIPS/X86).
- Mengubah Maksimal Menjadi 150 Proses : Untuk mengubahnya menjadi maksimal 150 proses, perlu mencari baris kode `NPROC` di file `include/kernel.h` dan mengubah nilainya:

2. [20 Poin] Jalankan kode sekuensial!

Jawaban: Untuk menjalankan fungsi secara sekuensial (berurutan), memanggil fungsi tersebut secara langsung di dalam fungsi `main()`. Fungsi kedua baru akan dieksekusi setelah fungsi pertama selesai sepenuhnya.

3. [20 Poin] Jalankan kode konkuren!

Jawaban: Untuk menjalankan fungsi secara konkuren (bersamaan), menggunakan fungsi bawaan Xinu yaitu `resume(create(...))`. Ini akan

membuat proses baru di dalam *ready list* dan menjadwalkannya secara bergantian menggunakan *scheduler* Xinu.

4. [50 Poin] Buatlah 2 proses produser dan konsumen. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```
int32 n = 0; //global variabel

void produser(void){
    int32 i;
    for (i=1; i<=1000; i++){
        n++;
    }
}

void konsumer(void){
    int32 i;
    for (i=1; i<=1000; i++){
        printf("Nilai dari n adalah %d\n",n);
    }
}
```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal).
Jelaskan mengapa hasilnya seperti itu!